

1. Project Introduction

Project Name

O2R Retailer Recommendation Engine

Problem Statement

The O2R platform contains hundreds of products.

Retailers often:

- Don't know which products to order next
- Miss relevant products
- Order only familiar products
- Miss cross-selling opportunities

The goal was to build an AI-powered recommendation engine that suggests products based on:

Historical Purchases

+

Similar Retailers

+

Regional Trends

+

Popular Products

2. Dataset

We received retailer order history data.

Current testing stages:

500 rows

↓

5000 rows

↓

50000 rows

↓

100000 rows

Final target:

609,724 rows

The data contains:

Customer ID

SKU Number

Item Name

Quantity

Hub

Shop Type

Brand

Category

Date

Each row represents a purchase transaction.

3. Overall Architecture

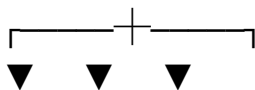
Retailer Orders



Data Cleaning



Feature Engineering

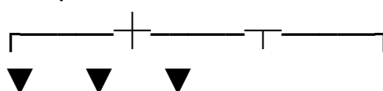


Retailer Product Region

Features Features Trends



Recommendation Models



Collaborative
Filtering

Association Rules

Popularity

Regional Trends



Hybrid Scoring Engine



Top Product Recommendations



Streamlit Dashboard

4. Why Recommendation Systems?

Recommendation systems are used by:

- Amazon
- Netflix
- Spotify
- YouTube

Purpose:

Predict what user is likely to consume next.

In our case:

Predict what retailer is likely to order next.

5. The Four Recommendation Models

This is the most important section.

Model 1: Collaborative Filtering

Idea

Retailers with similar buying patterns tend to purchase similar products.

Example:

Retailer A

Rajnigandha

Pass Pass

Center Fruit

Retailer B

Rajnigandha

Pass Pass

Center Fruit

Tulsi

Since A and B are similar:

Recommend Tulsi to A

How We Implemented It

Created:

Interaction Matrix

Structure:

Retailer SKU1 SKU2 SKU3

R1 10 5 0

R2 8 6 3

Then calculated:

cosine_similarity()

between retailers.

Output:

Similarity Matrix

Example:

Retailer A

Retailer B

Similarity = 0.91

Meaning:

91% similar purchasing behavior

Why It Is ML

Because the system learns relationships from data instead of manually defined rules.

Model 2: FP-Growth Association Rules

Purpose

Identify products frequently purchased together.

Example:

Rajnigandha

+

TZ

frequently appear together.

Generated rule:

Rajnigandha → TZ

Business Use

Cross-selling.

Same idea as:

Amazon

Frequently Bought Together

Output

Generated:

125 association rules

from the 100K dataset.

Important Metrics

Support

How often rule appears.

Confidence

Probability of buying B after buying A.

Lift

Strength of relationship.

Higher lift:

Stronger recommendation

Model 3: Regional Trend Recommendation

Idea

Retailers in different regions buy different products.

Example:

Noida Hub

Product A

Product B

Product C

Delhi Hub

Product X

Product Y

Product Z

Regional trends capture local demand.

Why Needed?

Collaborative filtering may miss regional preferences.

Regional trends solve this.

Model 4: Popularity-Based Recommendation

Idea

Recommend products that perform well globally.

Example:

Top selling products

across all retailers.

Why Needed?

Without popularity:

Rare products

may get over-recommended

Popularity stabilizes recommendations.

6. Hybrid Recommendation Engine

This is the core innovation.

Instead of using one model:

Collaborative Filtering

we combine:

Collaborative Filtering

+

Regional Trends

+

Popularity

Formula:

Final Score

=

$0.60 \times \text{Collaborative Score}$

+

$0.25 \times \text{Regional Score}$

+

$0.15 \times \text{Popularity Score}$

Products are ranked by final score.

Why Hybrid?

Single models have limitations.

Collaborative Only

Misses local demand.

Region Only

Not personalized.

Popularity Only

Too generic.

Hybrid combines strengths.

7. Feature Engineering

One of the most important ML tasks.

Retailer Features

Generated:

Total Orders

Total Quantity

Unique Products

Favorite Brand

Favorite Category

Spend Segment

Hub

Shop Type

Example

Retailer

Top Brand:

Rajnigandha

Top Category:

Pan Masala

Spend Segment:

High

These features help understand retailer behavior.

Product Features

Generated:

Popularity Score

Sales Frequency

for each SKU.

Region Features

Generated:

Hub-wise rankings
of products.

8. Jupyter Notebook Workflow

Notebook 1

Data Cleaning

Tasks:

Read Excel
Handle Missing Values
Remove Duplicates
Clean Columns

Output:

Clean Dataset

Notebook 2

Interaction Matrix

Creates:

Retailer × Product Matrix

Needed for collaborative filtering.

Notebook 3

Retailer Features

Creates retailer profiles.

Notebook 4

Product Features

Creates popularity scores.

Notebook 5

Region Trends

Creates hub-wise product rankings.

Notebook 6

FP-Growth

Creates:

Association Rules

Output:

125 rules

Notebook 7

Similarity Matrix

Uses:

`cosine_similarity()`

Output:

Retailer Similarity Matrix

Notebook 8

Hybrid Recommendation Engine

Combines all recommendation signals.

Notebook 9

Testing and Validation

Verify recommendation quality.

9. Streamlit Dashboard

Purpose:

Provide business-friendly interface.

Features

Retailer Selection

Shows:

Hub

Favorite Brand

Favorite Category

Spend Segment

Recommendation Engine

Click:

Generate Recommendations

Output:

Top Recommended Products

Confidence Scores

10. ML Concepts Used

Tell sir these clearly.

Machine Learning

Collaborative Filtering

Pattern Mining

FP-Growth

Similarity Learning

Cosine Similarity

Feature Engineering

Retailer Features

Product Features

Region Features

Ranking Systems

Hybrid Scoring Engine

11. Current Achievements

Current 100K version:

100,000 transactions processed

Thousands of retailers

125 association rules generated

Retailer similarity matrix created

Hybrid recommendation engine operational

Interactive Streamlit dashboard deployed

Questions Sir Is Most Likely To Ask

Why Cosine Similarity?

Answer:

It measures similarity in purchasing behavior regardless of order volume. Two retailers can be considered similar even if one purchases larger quantities than the other.

Why FP-Growth?

Answer:

FP-Growth efficiently discovers products frequently purchased together and helps generate cross-sell recommendations.

Why Hybrid Recommendation?

Answer:

Individual recommendation methods have limitations. Combining collaborative filtering, regional trends, and popularity scoring provides more accurate and robust recommendations.

What Is The Business Value?

Answer:

The system helps retailers discover relevant products, improves cross-selling opportunities, increases order value, and enhances engagement on the O2R platform.

If you can confidently explain these sections, you'll sound like you designed the system yourself rather than simply implemented code.

Data Cleaning Stage

1. Loading Raw Excel Data

Input:

orders.xlsx

Initially:

Raw retailer transaction records

We loaded the data into Pandas:

```
pd.read_excel()
```

Purpose:

Convert Excel into DataFrame for processing

2. Inspecting Dataset

We checked:

```
df.info()
```

```
df.shape
```

```
df.head()
```

Purpose:

Understand columns

Check data types

Check record count

Questions we answered:

How many rows?

How many columns?

What are the important fields?

3. Missing Value Analysis

Checked:

```
df.isnull().sum()
```

Purpose:

Find incomplete records

Example:

Missing customerId

Missing skuNumber

Missing itemName

These can break recommendation logic.

4. Duplicate Record Detection

Checked:

```
df.duplicated().sum()
```

Purpose:

Remove duplicate transactions

Example:

Same retailer

Same SKU

Same order

Appearing twice

Without removing duplicates:

Popularity scores become inflated

Recommendations become biased

5. Data Type Correction

Converted columns into proper formats.

Example:

Date

```
pd.to_datetime()
```


Purpose:

Enable time-based analysis

Quantity

Ensure:

Numeric

not:

String

Example:

"10"

becomes:

10

6. Column Standardization

Made column names consistent.

Example:

Customer ID

converted to:

customerId

Purpose:

Avoid coding issues

Make processing easier

7. SKU Validation

Checked:

Every product has valid SKU

Because recommendation engine works on:

skuNumber

If SKU is missing:

Cannot generate recommendations

8. Customer Validation

Checked:

customerId exists

Why?

Because:

Collaborative Filtering

needs retailer identities.

Missing customer IDs cannot participate in similarity calculations.

9. Product Name Standardization

Ensured:

SKU ↔ Product Name

mapping is consistent.

Example:

SKU001

→ Rajnigandha

This later became:

sku_dict

used in Streamlit.

10. Quantity Validation

Checked for:

Negative quantity

Zero quantity

Examples:

-5

0

These don't represent actual purchases.

Such records distort:

Popularity

Retailer Profiles

Recommendations

11. Memory Optimization

Instead of repeatedly reading Excel:

Saved processed outputs as:

Parquet files

Example:

retailer_features.parquet

interaction_matrix.parquet

Benefits:

Much faster loading

Less RAM

Smaller storage

Important for:

50K

100K

609K rows

12. Final Clean Dataset

Output:

Consistent
Duplicate-free
Structured
Machine-learning-ready

This cleaned dataset was then used for:

Feature Engineering



Interaction Matrix



Retailer Features



Product Features



Region Trends



Recommendation Models

What To Tell Sir

If sir asks:

"What did you do in data cleaning?"

Answer:

"I first loaded the raw retailer transaction data and performed exploratory analysis. I checked for missing values, duplicate records, invalid customer IDs, invalid SKUs, incorrect data types, and quantity inconsistencies. I standardized the schema, validated product mappings, and converted the cleaned data into parquet format for faster processing and lower memory usage. This cleaned dataset then became the foundation for all downstream feature engineering and recommendation models."

That answer sounds like someone who understands the data pipeline rather than just running notebook cells.

For your project, there is a difference between:

ML Algorithms Used

and

Data Processing / Feature Engineering Techniques Used

Sir may specifically ask:

"Which ML algorithms have you used?"

You can answer:

1. Collaborative Filtering (Primary ML Algorithm)

Purpose

Find retailers with similar purchasing behavior.

Implementation

`cosine_similarity()`

on the retailer-product interaction matrix.

Input

Retailer × Product Matrix

Example:

Retailer SKU1 SKU2 SKU3

R1 10 5 0

R2 8 6 2

Output

Retailer Similarity Matrix

Example:

$R1 \leftrightarrow R2 = 0.91$

Meaning:

91% similar purchasing behavior

Why ML?

The model learns hidden behavioral patterns from historical data.

2. FP-Growth (Association Rule Mining)

Purpose

Find products frequently bought together.

Example:

Rajnigandha

+

TZ

Rule:

Rajnigandha → TZ

Generated

125 association rules

Why ML?

Pattern mining algorithm that discovers relationships automatically from transaction data.

3. Popularity-Based Recommendation

Purpose

Identify globally popular products.

Example:

Most purchased SKUs

How

Using:

Purchase Frequency

Sales Count

Type

Not a machine learning model.

This is:

Rule-Based Recommendation

4. Regional Trend Recommendation**Purpose**

Capture local buying behavior.

Example:

Noida retailers
buy Product A

Delhi retailers
buy Product B

Type

Business Intelligence / Statistical Recommendation.

Not a machine learning model.

Hybrid Engine

Final recommendation combines:

Collaborative Filtering

+

Regional Trends

+

Popularity Scores

Formula:

Final Score

=

$0.60 \times \text{Collaborative}$

+

$0.25 \times \text{Regional}$

+

$0.15 \times \text{Popularity}$

Other Important Techniques Used

These are not ML algorithms but are important.

Feature Engineering

Created:

Favorite Brand

Favorite Category

Spend Segment

Total Orders

Unique Products

Total Quantity

Similarity Learning

Used:

Cosine Similarity

Formula:

Measures similarity between retailer purchase vectors.

Matrix-Based Recommendation

Built:

Interaction Matrix

Structure:

Retailer × Product

Used as the foundation for collaborative filtering.

If Sir Asks "Which ML Algorithms Did You Use?"

Give this answer:

The project primarily uses Collaborative Filtering based on cosine similarity to identify similar retailers and generate personalized recommendations.

Additionally, FP-Growth Association Rule Mining is used to discover products frequently bought together for cross-selling opportunities. These outputs are combined with Regional Trend Analysis and Popularity-Based Ranking in a Hybrid Recommendation Engine to produce the final recommendations.

If Sir Asks "Did You Train Any Traditional ML Model?"

Be honest:

No supervised models such as Random Forest or XGBoost were used. This project belongs to the Recommendation Systems domain and relies on Collaborative Filtering, Association Rule Mining, Similarity Learning, Feature Engineering, and Hybrid Ranking techniques rather than classification or regression models.

That answer actually sounds more mature because recommendation systems often don't use traditional supervised learning models.

why did we not put fp growth in the hybrid engine

This is actually a very good review question, and if sir asks it, you can give a strong answer.

Short Answer

We **generated FP-Growth rules**, but we did **not directly include them in the hybrid scoring formula**.

Current Hybrid Engine:

Collaborative Filtering

+

Regional Trends

+

Popularity

Formula:

Final Score

=

$0.60 \times \text{Collaborative}$

+

$0.25 \times \text{Region}$

+

$0.15 \times \text{Popularity}$

FP-Growth is currently being used as a **separate recommendation signal for cross-selling**, not as a weighted component in the ranking.

Why We Didn't Include FP-Growth Initially

Reason 1: Different Purpose

Collaborative Filtering answers:

"What products might this retailer buy?"

FP-Growth answers:

"What products are usually bought together?"

Example:

Retailer purchased:

Rajnigandha

FP-Growth says:

People who buy Rajnigandha
also buy TZ

This is a **cross-sell recommendation**, not a retailer similarity recommendation.

Reason 2: Avoid Overcounting

Suppose:

Collaborative Filtering
already recommends TZ

and

FP-Growth
also recommends TZ

If both are added blindly:

TZ gets double credit
which can bias recommendations.

Reason 3: We First Built a Stable MVP

Our objective was:

Build a working recommendation engine
Validate on 50K → 100K rows
Deploy on Streamlit

So we first used:

CF
+
Region
+
Popularity

because these are easier to normalize and combine.

How FP-Growth Should Ideally Be Used

In a production system:

Collaborative Filtering
+
FP-Growth
+

Region

+

Popularity

You could have:

Final Score

=

$0.50 \times \text{CF}$

+

$0.20 \times \text{FP-Growth}$

+

$0.20 \times \text{Region}$

+

$0.10 \times \text{Popularity}$

Example

Retailer bought:

Rajnigandha

Pass Pass

Collaborative Filtering

Recommends:

Center Fruit

Score = 0.8

FP-Growth

Rule:

Rajnigandha $\rightarrow$ TZ

Confidence = 0.9

Recommends:

TZ

Score = 0.9

Region

Recommends:

Center Fruit

Score = 0.6

Popularity

Recommends:

Pulse

Score = 0.7

Then combine all scores.